



5

Crafting Reusable Components

The aim of this chapter is to show you how to implement React components that serve more than just one purpose. After reading this chapter, you'll feel confident about how to compose application features.

The chapter starts with a brief look at HTML elements and how they work in terms of helping to implement features versus having a high level of utility. Then, you'll see the implementation of a **monolithic component** and discover the issues that it will cause down the road. The next section is devoted to re-implementing the monolithic component in such a way that the feature is composed of smaller components.

Finally, the chapter ends with a discussion of rendering trees of React components and gives you some tips on how to avoid introducing too much complexity as a result of decomposing components. I'll close the final section by reiterating the con-

cept of high-level feature components versus utility components.

The following topics will be covered in this chapter:

- Reusable HTML elements
- The difficulty with monolithic components
- Refactoring component structures
- Render props
- Rendering component trees

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter05>.

Reusable HTML elements

Let's think about HTML elements for a moment. Depending on the type of HTML element, it's either feature-centric or utility-centric. Utility-centric HTML elements are more reusable than feature-centric HTML elements. For example, consider the `<section>` element. This is a generic element that can be used just about anywhere but its primary purpose is to compose the structural aspects of a feature: the outer shell of the feature

and the inner sections of the feature. This is where the `<section>` element is most useful.

On the other side of the fence, you have elements such as `<p>`, `<span>`, and `<button>`. These elements provide a high level of utility because they're generic by design. You're supposed to use `<button>` elements whenever you have something that's clickable by the user, resulting in an action. This is a level lower than the concept of a feature.

While it's easy to talk about HTML elements that have a high level of utility versus those that are geared toward specific features, the discussion is more detailed when data is involved. HTML is static markup; React components combine static markup with data. The question is, how do you make sure that you're creating the right feature-centric and utility-centric components?

The aim of this chapter is to find out how to go from a monolithic React component that defines a feature to a smaller feature-centric component combined with utility components.

The difficulty with monolithic components

If you could implement just one component for any given feature, it would simplify your job. At the very least, there wouldn't be many components to maintain, and there wouldn't be many communication paths for data to flow through because everything would be internal to the component.

However, this idea doesn't work for a number of reasons. Having monolithic feature components makes it difficult to coordinate any kind of team development effort, such as **version control**, **merge conflicts**, and **parallel development**. The bigger the monolithic components become, the more difficult they are to refactor into something better later on.

There's also the problem of feature overlap and feature communication. Overlap happens because of similarities between features; it's unlikely that an application will have a set of features that are completely unique to one another. That would make the application very difficult to learn and use.

Component communication essentially means that the state of something in one feature will impact the state of something in another feature. State is difficult to deal with, and even more so when there is a lot of state packaged up in a monolithic component.

The best way to learn how to avoid monolithic components is to experience one firsthand. You'll spend the remainder of this section implementing a monolithic component. In the following section, you'll see how this component can be refactored into something a little more sustainable.

The JSX markup

The **monolithic component** we're going to implement is a feature that lists articles. It's just for illustrative purposes, so we don't want to go overboard on the size of the component. It'll be simple yet monolithic. The user can add new items to the list, toggle the summary of items in the list, and remove items from the list.

Here is the JSX markup of the component:

```
<section>
  <header>
    <h1>Articles</h1>
    <input placeholder="Title" value={title} onChange={onChangeTitle} />
    <input
      placeholder="Summary"
      value={summary}
      onChange={onChangeSummary}
    />
    <button onClick={onClickAdd}>Add</button>
  </header>
```

```
<article>
  <ul>
    {articles.map((i) => (
      <li key={i.id}>
        <a
          href={`#${i.id}`}
          title="Toggle Summary"
          onClick={() => onClickToggle(i.id)}
        >
          {i.title}
        </a>
        &nbsp;
        <button
          href={`#${i.id}`}
          title="Remove"
          onClick={() => onClickRemove(i.id)}
        >
          &#10007;
        </button>
        <p style={{ display: i.display }}>{i.summary}</p>
      </li>
    ))}
  </ul>
</article>
</section>
```

This is definitely more JSX than is necessary in one place. We'll improve on this in the following section, but for now, let's implement the initial state for this component.

Initial state

Now, let's look at the initial state of this component:

```
const [articles, setArticles] = React.useState([
  {
    id: id.next(),
    title: "Article 1",
    summary: "Article 1 Summary",
    display: "none",
  },
  {
    id: id.next(),
    title: "Article 2",
    summary: "Article 2 Summary",
    display: "none",
  },
]);
const [title, setTitle] = React.useState("");
const [summary, setSummary] = React.useState("");
```

The state consists of an array of `articles`, a `title` string, and a `summary` string. Each article object in the `articles` array has several string fields to help render the article and an `id` field, which is a number. The number is generated by `id.next()`.

Let's take a look at how this works:

```
const id = (function* () {  
  let i = 1;  
  while (true) {  
    yield i;  
    i += 1;  
  }  
})();
```

The `id` constant is a generator. It is created by defining an inline generator function and calling it right away. This generator will yield numbers infinitely. So, calling `id.next()` the first time returns `1`, the next is `2`, and so on. This simple utility will come in handy when it's time to add new articles and we need a new unique ID.

Event handler implementation

At this point, you have the initial state and the JSX of the component. Now, it's time to implement the event handlers:

```
const onChangeTitle = useCallback((e) => {  
  setTitle(e.target.value);  
}, []);  
const onChangeSummary = useCallback((e) => {  
  setSummary(e.target.value);  
}, []);
```


The `onChangeTitle()` and `onChangeSummary()` methods use the hook's `setState()` to update the `title` and `summary` state values, respectively. The new values come from the `target.value` property of the `event` argument, which is the value that the user types into the text input:

```
const onClickAdd = useCallback(() => {
  setArticles((state) => [
    ...state,
    {
      id: id.next(),
      title: title,
      summary: summary,
      display: "none",
    },
  ]);
  setTitle("");
  setSummary("");
}, [summary, title]);
```

The `onClickAdd()` method adds a new article to the `articles` state. This state value is an array. We use the spread operator to build a new array from the existing array (`[...state]`), and the new object gets added to the end of the new array. The reason we're building a new array and passing it to `setArticles()` is so that there are no surprises. In other words, we're treating state values as immutable so that other code that up-

dates the same state doesn't accidentally cause problems. Next, we'll use a handler to remove the article:

```
const onClickRemove = useCallback((id) => {
  setArticles((state) =>
    state.filter((article) => article.id !== id)
  );
}, []);
```

The `onClickRemove()` method removes the article with the given ID from the `articles` state. It does this by calling `filter()` on the array, which returns a new array, so the operation is immutable. The filter removes the object with the given ID:

```
const onClickToggle = useCallback((id) => {
  setArticles((state) => {
    const articles = [...state];
    const index = articles.findIndex((article) => article.id === id);
    articles[index] = {
      ...articles[index],
      display: articles[index].display ? "" : "none",
    };
    return articles;
  });
}, []);
```

The `onClickToggle()` method toggles the visibility of the article with the given ID. We carry out two immutable operations in this method. First, we build a new `articles` array. Then, based on the index of the given ID, we replace the article object at the index with a new object. We use the object spread operator to fill in the properties (`{...articles[index]}`), and then the display property value is toggled based on the existing display value.

Here's a screenshot of the output rendered:

Articles

Title	Summary	Add
-------	---------	-----

- [Article 1](#) ☐
- [Article 2](#) ☐

Figure 5.1: Rendered articles

At this point, we have a component that does everything that we need our feature to do. However, it's monolithic and difficult to maintain. Imagine if we had other places in our app that use the same pieces of `MyFeature`. They have to re-invent them because they cannot be shared. In the following section,

we'll work on breaking down `MyFeature` into smaller reusable components.

Refactoring component structures

You have a monolithic feature component: *now what?* Let's make it better.

In this section, you'll learn how to take the feature component that you just implemented in the preceding section and split it into more maintainable components. You'll start with the **JSX**, as this is probably the best refactor starting point. Then, you'll implement new components for the feature.

Next, you'll make these new components functional instead of class-based. Finally, you'll learn how to use **render props** to reduce the number of direct component dependencies in your application, and how to remove classes entirely by using hooks to manage state within functional components.

Starting with the JSX

The **JSX** of any **monolithic component** is the best starting point for figuring out how to refactor it into smaller compo-

nents. Let's visualize the structure of the component that we're currently refactoring:

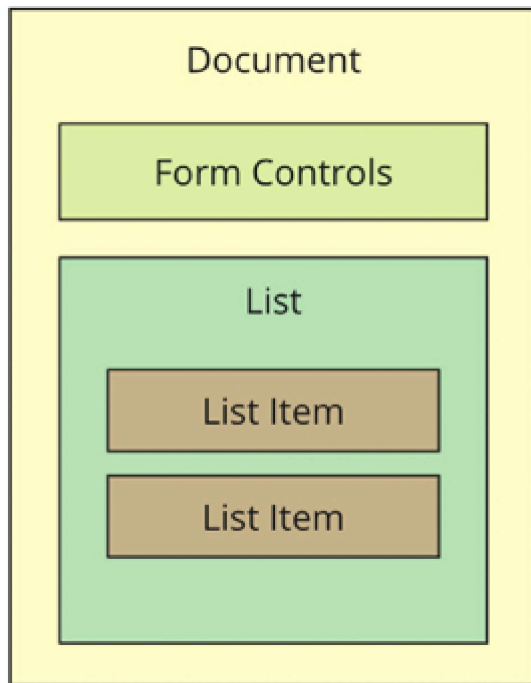


Figure 5.2: Visualization of the JSX that makes up a React component

The top part of the JSX is the form controls, so this could easily become its own component:

```
<header>
  <h1>Articles</h1>
  <input
```

```
    placeholder="Title"
    value={title}
    onChange={onChangeTitle} />
<input
  placeholder="Summary"
  value={summary}
  onChange={onChangeSummary} />
<button onClick={onClickAdd}>Add</button>
</header>;
```

Next, you have the list of articles:

```
<ul>
  {articles.map((i) => (
    <li key={i.id}>
      <a
        href={`#${i.id}`}
        title="Toggle Summary"
        onClick={() => onClickToggle(i.id)}
      >
        {i.title}
      </a>
      &nbsp;
      <button
        href={`#${i.id}`}
        title="Remove"
        onClick={() => onClickRemove(i.id)}
      >
        &#10007;
```

```
        </button>
        <p style={{ display: i.display }}>{i.summary}</p>
      </li>
    )}
  </ul>
```

Within this list, there's potential for an article component, which would be everything in the `<li>` tag. Let's try building this next.

Implementing an article list component

Here's what the ArticleList component implementation looks like:

```
function ArticleList({ articles, onClickToggle, onClickRemove }) {
  return (
    <ul>
      {articles.map((i) => (
        <li key={i.id}>
          <a
            href={`#${i.id}`}
            title="Toggle Summary"
            onClick={() => onClickToggle(i.id)}
          >
            {i.title}
          </a>
          &nbsp;

```

```
    <button
      href={`#${i.id}`}
      title="Remove"
      onClick={() => onClickRemove(i.id)}
    >
      &#10007;
    </button>
    <p style={{ display: i.display }}>{i.summary}</p>
  </li>
))}
</ul>
);
}
```

We're taking the relevant JSX out of the monolithic component and putting it here. Now, let's see what the feature component of JSX looks like:

```
<section>
  <header>
    <h1>Articles</h1>
    <input placeholder="Title" value={title} onChange={onChangeTitle} />
    <input
      placeholder="Summary"
      value={summary}
      onChange={onChangeSummary}
    />
    <button onClick={onClickAdd}>Add</button>
  </header>
```



```
<ArticleList
  articles={articles}
  onClickRemove={onClickRemove}
  onClickToggle={onClickToggle}
/>
</section>
```

The list of articles is now rendered by the `ArticleList` component. The list of articles to render is passed to this component as a property along with two of the event handlers.



Why are we passing event handlers to a child component? The reason is so that the `ArticleList` component doesn't have to worry about the state or how the state changes. All it cares about is rendering content and making sure the appropriate event callbacks are hooked up to the appropriate DOM elements. This is a container component concept that I'll expand upon later in this chapter.

Now that we have an `ArticleList` component, let's see whether we can further break it down into smaller reusable components.

Implementing an article item component

After implementing the article list component, you might decide that it's a good idea to break this component.

Another way to look at it is this: if it turns out that we don't actually need the item as its own component, this new component doesn't introduce much indirection or complexity.

Without further ado, here's the article item component:

```
function ArticleItem({ article, onClickRemove }) {
  const [isOpen, setIsOpen] = React.useState(article.display !== "none");
  const onClickToggle = React.useCallback(() => {
    setIsOpen((state) => !state);
  }, []);
  return (
    <li>
      <a href={`#${article.id}`} title="Toggle Summary" onClick={onClickToggle}>
        {article.title}
      </a>
      &nbsp;
      <button
        href={`#${article.id}`}
        title="Remove"
        onClick={() => onClickRemove(article.id)}
      >
        ✖
      </button>
      <p style={{ display: isOpen ? "block" : "none" }}>{article.summary}</p>
    </li>
  );
}
```

```
);  
}
```

Essentially, the component remains unchanged except for one enhancement: we have relocated the logic for expanding and collapsing the article to the `ArticleItem` component, which offers several advantages. Firstly, we reduced the original `MyFeature` component, since it doesn't need to know when we hide or expand an article at all. Secondly, we have improved the performance of the application due to the fact that when expanding an article, we no longer recreate the array of articles using the spread operator but only change the state locally. As a result, when expanding the article, the list of articles remains the same and React does not re-render the page, but only one component is re-rendered.

Here's the new `ArticleItem` component being rendered by the `ArticleList` component:

```
function ArticleList({ articles, onClickRemove }) {  
  return (  
    <ul>  
      {articles.map((article) => (  
        <ArticleItem  
          key={article.id.value}  
          article={article}  
          onClickRemove={onClickRemove}    )  
    }  
    </ul>  
  )  
}
```

```
        />  
      )}}  
    </ul>  
  );  
}
```

Do you see how this list just maps the list of articles? What if you wanted to implement another article list that does some filtering too? If so, it's beneficial to have a reusable `ArticleItem` component. Next, we'll move the add article markup into its own component.

Implementing an AddArticle component

Now that we're done with the article list, it's time to think about the form controls used to add a new article. Let's implement a component for this aspect of the feature:

```
function AddArticle({  
  name,  
  title,  
  summary,  
  onChangeTitle,  
  onChangeSummary,  
  onClickAdd,  
}) {  
  return (  
    <section>
```

```
    <h1>{name}</h1>
    <input placeholder="Title" value={title} onChange={onChangeTitle} />
    <input placeholder="Summary" value={summary} onChange={onChangeSummary} />
    <button onClick={onClickAdd}>Add</button>
  </section>
);
}
```

Now, our feature component only needs to render

`<AddArticle>` and `<ArticleList>` components:

```
<section>
  <AddArticle
    name="Articles"
    title={title}
    summary={summary}
    onChangeTitle={onChangeTitle}
    onChangeSummary={onChangeSummary}
    onClickAdd={onClickAdd}
  />
  <ArticleList articles={articles} onClickRemove={onClickRemove} />
</section>
```

The focus of this component is on the feature data, while it defers to other components for rendering UI elements. In the next section, we'll look at how render props make it possible to pass components around as properties instead of directly importing them as dependencies.

Render props

Imagine implementing a feature that is composed of several smaller components, like what you've been working on in this chapter. The `MyFeature` component depends on `ArticleList` and `AddArticle`. Now, imagine using `MyFeature` in different parts of your application where it makes sense to use a different implementation of `ArticleList` or `AddArticle`. The fundamental challenge is substituting one component for another.

Render props are a nice way to address this challenge. The idea is that you pass a property to your component whose value is a function that returns a component to render. This way, instead of having the feature component directly depend on its child components, you can configure them as you like; they pass them in as render prop values. Let's look at an example. Instead of having `MyFeature` directly depend on `AddArticle` and `ArticleList`, you can pass them as render props. Here's what the `MyFeature` looks like when it's using render props to fill in the holes where `add` used to be:

```
<section>
  {addArticle({
    title,
    summary,
    onChangeTitle,
    onChangeSummary,
```

```
      onClickAdd,  
    }}}  
    {articleList({ articles, onClickRemove })}  
  </section>
```

The `addArticle()` and `articleList()` functions are called with the same property values that would have been passed to `<AddArticle>` and `<ArticleList>`, respectively. The difference now is that this module no longer imports `AddArticle` or `ArticleList` as dependencies.

Now, let's take a look at the `main.js` file where `<MyFeature>` is rendered:

```
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(  
  <MyFeature  
    addArticle={({  
      title,  
      summary,  
      onChangeTitle,  
      onChangeSummary,  
      onClickAdd,  
    }) => (  
      <AddArticle  
        name="Articles"  
        title={title}  
        summary={summary}
```

```
        onChangeTitle={onChangeTitle}
        onChangeSummary={onChangeSummary}
        onClickAdd={onClickAdd}
      />
    )}
    articleList=(({ articles, onClickRemove }) => (
      <ArticleList articles={articles} onClickRemove={onClickRemove} />
    ))
  />
);
```

There's a lot more going on here now than there was when it was just `<MyFeature>` being rendered. Let's break down why that is. Here is where you pass the `addArticle` and `articleList` render props. These prop values are functions that accept argument values from `MyComponent`. For example, the `onClickRemove()` function comes from `MyFeature` and is used to change the state of that component. You can use the **render prop** function to pass this to the component that will be rendered, along with any other values. The return value of these functions is what is ultimately rendered.

In this section, you learned that by passing render property values: functions that render JSX markup: you can avoid hard-coding dependencies in places where you might want to share functionality. Passing a different property value to a compo-

nent is usually easier than changing the dependencies used by a given module.

Rendering component trees

Let's take a moment to reflect on what we've accomplished so far in this chapter. The feature component that was once monolithic ended up focusing almost entirely on the state data. It handled the initial state and handled transforming the state and it would handle network requests that fetch state, if there were any. This is a typical container component in a React application, and it's the starting point for data.

The new components that you implemented to better compose the feature were the recipients of this data. The difference between these components and their container is that they only care about the properties that are passed into them at the time they're rendered. In other words, they only care about data snapshots at a particular point in time. From here, these components might pass the property data into their own child components as properties. The generic pattern for composing React components is as follows:

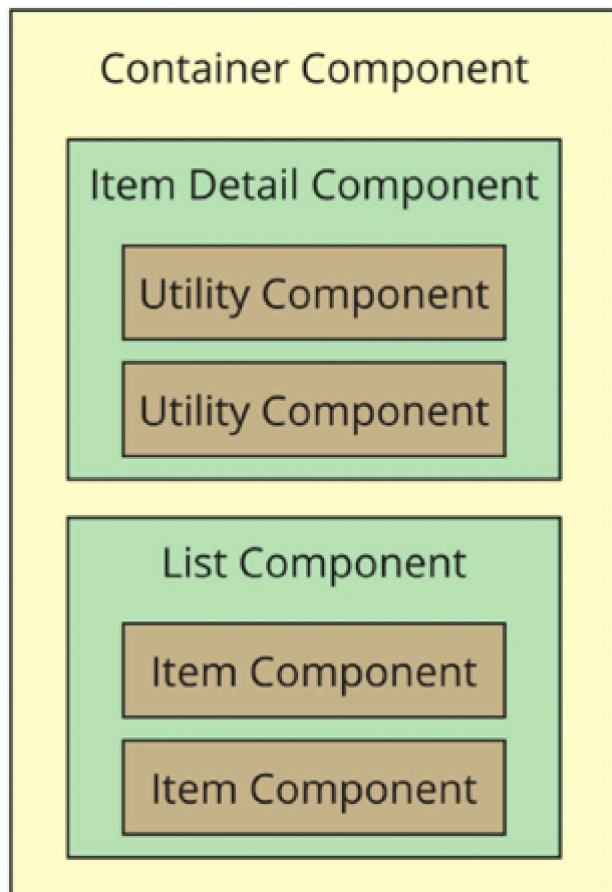


Figure 5.3: A pattern for composing larger React components from smaller components

The **container component** will typically contain one direct **child**. In this diagram, you can see that the container has either an **item detail component** or a **list component**. Of course, there will be variations in these two categories, as every application is different. This generic pattern has three lev-

els of component composition. Data flows in one direction from the **container** all the way down to the **utility** components.

Once you add more than three layers, the application architecture becomes difficult to comprehend. There will be the odd case where you'll need to add four layers of React components but, as a rule of thumb, you should avoid this.

Feature components and utility components

In the **monolithic component** example that we have worked on in this chapter, you started with a single component that was entirely focused on a feature. This means that the component has very little utility elsewhere in the application.

The reason for this is that top-level components deal with the application state. **Stateful components** are difficult to use in any other context. As you refactored the monolithic feature component, you created new components that moved further away from the data. The general rule is that the further your components move from stateful data, the more utility they have because their property values could be passed in from anywhere in the application.

Summary

This chapter was about avoiding a monolithic component design. However, monoliths are often a necessary starting point in the design of any React component.

You began by learning about how the different HTML elements have varying degrees of utility. Next, you learned about the issues with monolithic React components and walked through the implementation of a monolithic component.

Then, you spent several sections learning how to refactor the monolithic component into a more sustainable design. From this exercise, you learned that container components should only have to think in terms of handling state, while smaller components have more utility because their property values can be passed from anywhere. You also learned that you could use render props for better control over component dependencies and substitution.

In the next chapter, you'll learn about the component props validation and type checking.